

Modal Execution Model

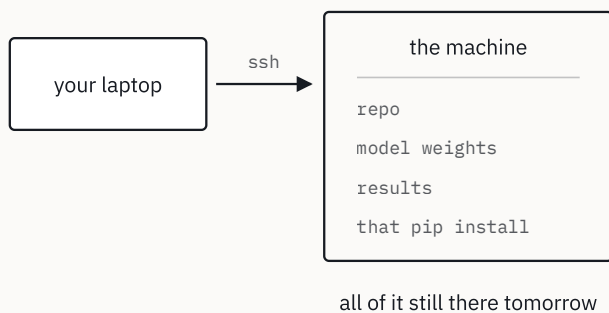
There is no machine. Nothing is cloned, nothing is left behind, and the part you keep has to be declared. A map of where code, weights, GPUs and results actually live.

01 The mental model

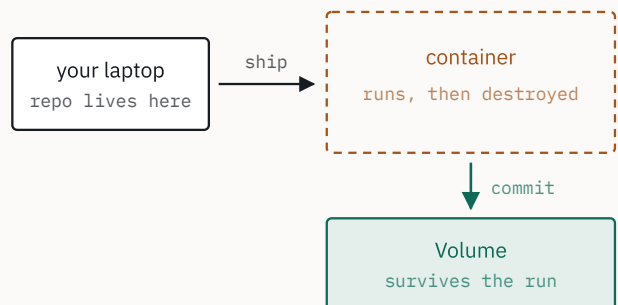
Coming from RunPod or Lambda, the expectation is a box: you rent it, SSH in, clone the repo, install things, run them, and everything you did is still sitting there tomorrow.

Modal has no box. Your code lives on your laptop and is shipped to a container that executes it and is then destroyed. You never log in to anything. You invoke a function locally and it runs remotely.

A RENTED BOX



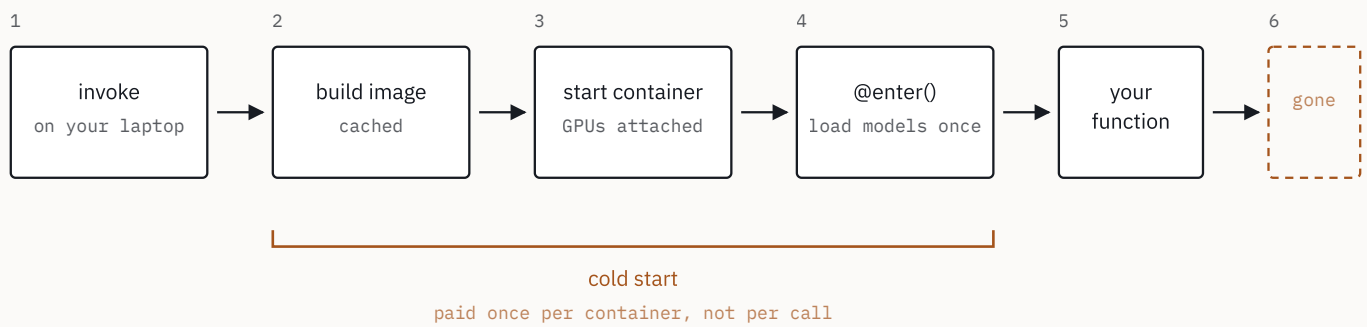
MODAL



The difference is not where compute happens, it is what state outlives the run. On a rented box everything persists by default. On Modal nothing does, unless you declared a Volume for it.

02 What happens when you type `modal run`

One command triggers a fixed sequence. Knowing which steps are cached and which repeat is most of what you need to reason about cost and cold starts.

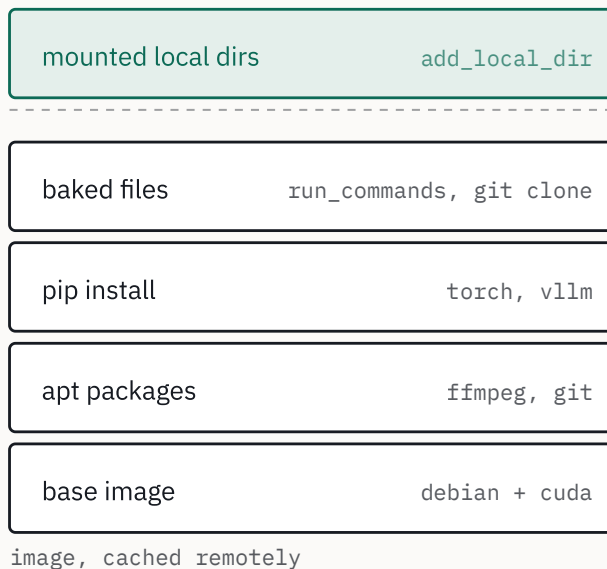


Steps 2 through 4 are the cold start. For a long-running benchmark job this is harmless: it is paid once, before any measured trial. For a live service it is the thing you fight, which is why `min_containers` exists.

03 The image: baked versus mounted

An image is a stack of layers built from a declaration in your Python file, not from a Dockerfile you maintain by hand (though it can start from one). The layers are cached and content-addressed, so a rebuild only replays from the first thing that changed.

The distinction that matters day to day is whether a file is *baked into a layer* or *mounted at run time*. Baked files are part of the image, so changing one invalidates that layer and everything above it. Mounted files are synced on every invocation and never trigger a rebuild.



edit a mounted file
and nothing rebuilds

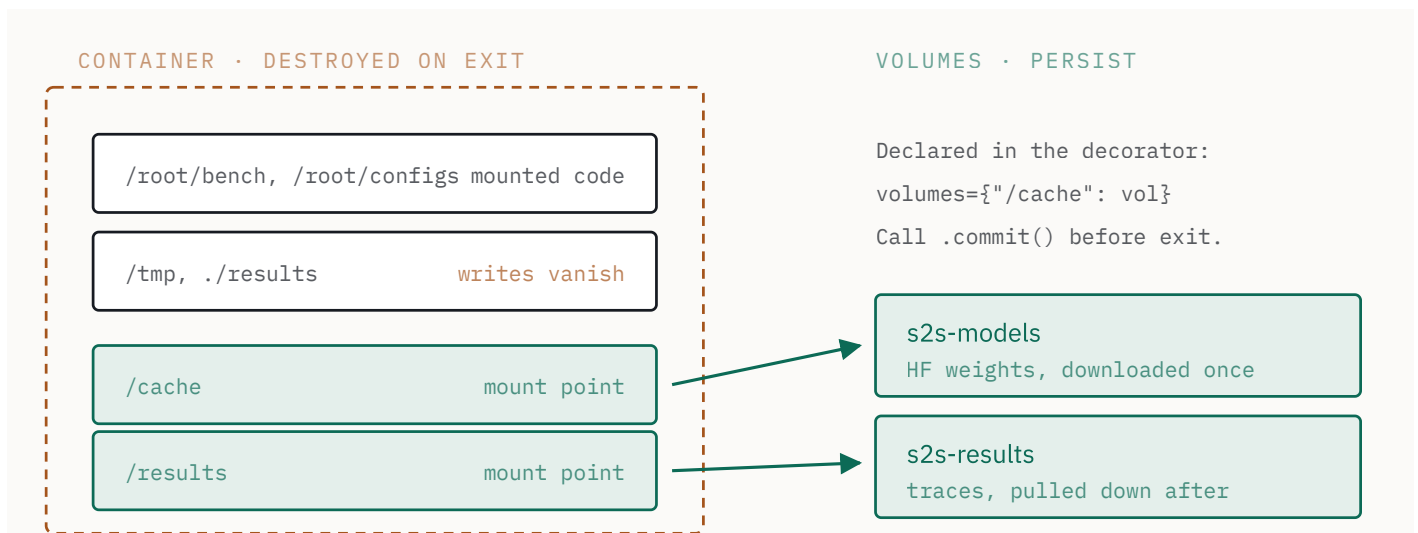
edit a baked file
and this layer rebuilds,
along with everything above

Put actively edited code in mounted directories and stable dependencies in baked layers. Cloning your own repo inside the image is the common mistake: every commit invalidates the layer and you wait through a rebuild to test a one-line change.

```
image = (  
    modal.Image.from_dockerfile("Dockerfile")    # baked, stable  
    .env({"HF_HOME": "/cache/hf"})  
    .add_local_dir("bench",    "/root/bench")    # mounted, edited daily  
    .add_local_dir("services", "/root/services")  
    .add_local_dir("configs",  "/root/configs")  
)
```

04 Filesystem: what survives

Three separate worlds, and code running inside the container can only see two of them. Your laptop's filesystem is not one of them, which catches people out when a script writes results next to itself and they vanish.



A Volume is a named, durable filesystem you mount into the container at a path you choose. Everything outside those mount points is scratch space that disappears the moment the function returns.

LOCATION	SURVIVES THE RUN	WHAT BELONGS THERE
your laptop	not visible remotely	Source of truth for code. Shipped in, never written back to.
image layers	cached	Dependencies, system packages, anything stable.
container FS	destroyed	Scratch only. Writing results here loses them.
Volume	persists	Model weights, datasets, output traces.
Secret	persists	Tokens, injected as env vars. Never bake these into an image.

05 Scope: which lines run where

The single most confusing thing about Modal is that one file executes in two places. Module-level code runs on your laptop when you invoke, *and again* inside the container when it imports your module. Function bodies run only remotely.

This is why a bare `import torch` at the top of the file breaks a laptop that has no CUDA. Guard heavy imports so they only resolve inside the container.

<pre>import modal app = modal.App("s2s-bench") image = modal.Image.from_dockerfile("Dockerfile")</pre>	Both. Runs locally to define the app, and again in the container on import.
<pre>with image.imports(): import torch, vllm</pre>	Container only. Your laptop never needs these installed.
<pre>@app.function(gpu="A10G:4", volumes={...}) def run_bench(configs): ...</pre>	Container only. The body never executes on your machine.
<pre>@app.local_entrypoint() def main(): run_bench.remote([...])</pre>	Laptop only. The dispatcher. <code>.remote()</code> is the boundary crossing.

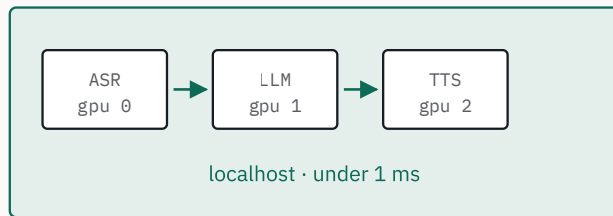
`.remote()` blocks until the remote function returns and hands back its value, serialized. `.spawn()` fires and forgets, returning a handle. `.map()` fans one function across many inputs, each in its own container, which is where Modal genuinely shines.

06 GPUs and container topology

A GPU is requested per function, as a string. `gpu="A10G"` gives that container one card; `gpu="A10G:4"` gives the same container four. That colon is the whole decision for a multi-stage pipeline.

Splitting stages across separate Modal functions puts them in separate containers on separate machines. Everything they exchange crosses a network. For a pipeline being measured in milliseconds, that added hop is not a detail, it is the measurement.

ONE CONTAINER · `gpu="A10G:4"`



stages pinned with `CUDA_VISIBLE_DEVICES`

FOUR FUNCTIONS · ONE GPU EACH



network hop on every exchange
milliseconds each way, and variable

fine for batch work
fatal for latency measurement

For a latency benchmark the left shape is the only correct one: it reproduces a rented multi-GPU box exactly, with services as subprocesses talking over localhost. The right shape is the natural reading of "one service per stage" and it would put the thing being measured inside the measurement.

WORTH KNOWING BEFORE YOU COMMIT

Modal is multi-tenant. Your GPUs are dedicated to your container, but the host CPU and PCIe are not necessarily, and jitter from a co-tenant is indistinguishable from jitter in your own pipeline.

Run a short pilot and check how many trials your feeder-lag gate discards. A low rate means the host is quiet enough to trust; a high one means the measurement sweep belongs on a dedicated pod even if development stays here.

07 The shape for a benchmark

A benchmark is a job, not a service: one long invocation that sets everything up, runs every config, writes its output to durable storage, and exits. That plays to Modal's strengths, because the cold start is paid once before any measured trial and the container's disposability stops mattering.

```

app = modal.App("s2s-latency-bench")
cache = modal.Volume.from_name("s2s-models", create_if_missing=True)
results = modal.Volume.from_name("s2s-results", create_if_missing=True)

@app.function(
    gpu="A10G:4", # one container, four cards
    image=image,
    volumes={"/cache": cache, "/results": results},
    timeout=24 * 3600, # default is far too short
    secrets=[modal.Secret.from_name("huggingface")],
)
def run_bench(configs):
    procs = [spawn(m, dev) for dev, m in enumerate(SERVICES)]
    try:
        for cfg in configs:
            subprocess.run(["python", "-m", "bench.runner",
                           "--config", cfg,
                           "--out", f"/results/{name(cfg)}.jsonl"], check=True)
    finally:
        for p in procs: p.terminate()
    results.commit() # without this, the traces are gone

```

The four things that bite

- **Forgetting `.commit()`**. The function succeeds, the traces are written, and nothing is there afterwards.
- **Leaving `HF_HOME` at its default**. Every run re-downloads every checkpoint, which is slow and burns credits on nothing.
- **The default timeout**. A real-time feeder streams at 1x, so a sweep takes wall-clock hours. Raise it explicitly.
- **Multi-GPU queueing**. `A10G:4` waits for capacity where a single `A10G` starts immediately. Pin the same class for every reported run regardless.

When Modal is the wrong tool

If your instinct is to clone a repo and run it, Modal is asking you to restructure how you deploy in exchange for credits. A rented pod matches that instinct exactly and gives a quieter host for jitter-sensitive work. Modal repays the restructuring when you want to

fan a function across many containers at once, or when the workload is genuinely bursty. For one long measurement sweep on four GPUs, it is a fair trade at best.

Reference notes for the s2s-latency-bench project. API surfaces move; treat the shapes here as the model and check the current docs for exact signatures.